Repairing a Conference Schedule

Student: Michael Warner
Private repository: [(https://github.com/michaelwarner27/AI4300/tree/main/U0-HW-04)]
Access: [Confirm `fractal13` has read access]
Submitted commit: [(https://github.com/michaelwarner27/AI4300/commit/ea8406154755065debd1cf469b2b9969079ba09f)]

1. PEAS Task-Environment Assessment

Agent and Task Boundary

The agent takes an evaluation from the fixed benchmark and tries a series of steps to get a better evaluation, one with 0 hard conflicts. The agent's job is to return the final schedule object not the steps it took to get there. The fixed benchmark determines how hard and soft conflicts are counted and is in charge of scoring each given schedule.

Performance Measure

Hard feasibility requires that there is no more than 1 speaker in a room at a given time, that a speaker is only giving 1 lecture at a given time, and that an expected audience only has 1 talk happening at any given time. Soft quality is defined by speakers' preferred time slots and using as few rooms as possible. The soft quality at first feasibility is the soft quality of the schedule as soon as the agent finds a schedule with 0 hard conflicts. This may not be the most optimal schedule but it is a working schedule that has been found. Budget limits our agent's number of moves so that it does not end up in infinite loops. An example of an unacceptable outcome is a schedule that has speaker A in two

classrooms at 9:30. Soft quality cannot override hard feasibility because using as fewer rooms doesn't matter if that causes multiple speakers to be lecturing in the same room at the same time.

Environment

Our conference problem has an environment that includes a set of talks each with a speaker and audience. Theses need to get sorted into our conference rooms during available time slots. Additionally every speaker has preferred time slots for their lecture. Something that is missing from our problem is additional room constraints, like how many people can fit in the room. The talk data type actually accounts for this, listing anticipated audience size, but we don't check this against the room size.

Actuators

The agents are only able to change a single talk's placement at a time. This allows us to evaluate every schedule rather than willy nilly changing things around. The agent is only able to change the room and time a talk is held. It is unable to change the speaker, audience, or time preferences. An LLM requested move can be anything, it gets validated if it is a legal move for example is only changing a single talk's placement, and it only gets applied if that movement improves the score.

Sensors

The agents are only given the current state and its evaluated score. They are then able to send possible new schedules to the evaluator to be scored. An information limitation is that we don't track previous states. This causes some inefficiencies because we may revisit a state we've been to rather than choosing a new state that has the same evaluated score.

Environment Classification

Dimension	Classification	Benchmark-specific justification
Observability	[Fully observable]	[The agents are able to see the entire current state.]
Outcomes	[Deterministic]	[Because we use seeded randomness for choosing a move when they score the same this all of the agents actions are deterministic.]
Temporal structure	[Sequential]	[Because the agent sends for evaluation after every move and breaks if the schedule has more hard conflicts this environment is sequential.]
Dynamics	[Static]	[The environment only changes when our agent acts on it. The number of talks or where they are assigned doesn't change while the agent is solving.]
State and actions	[Discrete]	[There is a fixed number of possible schedules and our budget constraint ensures the agent doesn't get stuck in a infinite loop between two states.]
Multiplicity	[Single Agent]	[Only our agent is working on this schedule.]

Agent-Environment Cycle and Implementation Mapping

Agent: minconflicts Scenario: conference-1

PEAS element	Code or data interface	Evidence or metric

Performance measure	[Hard violations: 0, Soft penalty = 5]	[There are no hard conflicts but 5 soft penalties of speakers not getting their preferred time slots and the number of rooms used by each track.]
Environment	[_talk("T1", "Ada", "AI", 140, all_slots, ("09:00",), ("recording",)),	

```
    _talk("T2", "Bo", "Systems", 70, all_slots, ("10:30",
    _talk("T3", "Ada", "AI", 40, all_slots, ("10:30",), (
    _talk("T4", "Cy", "Systems", 35, ("09:00", "13:00"),
    _talk("T5", "Dee", "AI", 65, all_slots, ("13:00",)),
    _talk("T6", "Eli", "Practice", 30, ("10:30", "13:00")
```

| Actuators | [TraceStep(step=1, request="T3->Placement(room='Mesa', slot='13:00')", outcome='applied', hard_violations=3, soft_penalty=5) TraceStep(step=2, request="T1->Placement(room='Auditorium', slot='10:30')", outcome='applied', hard_violations=1, soft_penalty=6) TraceStep(step=3, request="T4->Placement(room='Canyon', slot='13:00')", outcome='applied', hard_violations=0, soft_penalty=5)] | [These are the steps that the agent took to get to a valid schedule.] | | Sensors | [hv=3, sp=5; hv=1, sp=6; hv=0, sp=5] | [After each step the agent the schedule to get evaluated as seen in the actuators above] |

2. CSP Model and Evaluator

Variables, Domains, and Unary Filtering

The Variables tracked are talk objects which contain a placement pair of (room, slot). The domain keeps track of room capacity, equipment, and availability. Unary filtering requires every schedule to be structurally valid. Any schedule that exists outside of the bounds of the given

domain is filtered out.

Structural Validity, Feasibility, and Goal

Structural validity is concerned about a schedule fitting the domain. Does the room a talk assigned to meet that talk's requirements. Does the room fit enough people and have the right equipment. Hard feasibility is concerned with overlap of talks and speakers. The final solution is a final schedule because the step to get there are useless to us. While the agent is only able to move one talk at a time, as the human overlords we can apply an entire schedule at once. A valid feasible schedule is all we need, not the path the agent took to find it.

Hard Constraints and Soft Penalty

[Define every count and the exact soft formula. Include a hand-checked example.] Hard penalty is incremented every time a hard conflict is found, two talks in the same room at the same time, a speaker lecturing in twice in the same time slot, and an audience having multiple talks going at the same time. Every conflict is counted individually so a pair of talks that has multiple of these violations counts each violation. Soft penalty is incremented when a speaker's preferred slot set doesn't have a talk assigned in one of those preferred slots. Plus the number of rooms that each track uses -1. Every track has to use at least one room.

Hand-checked example — conference-1 initial schedule: AI generated table

Talk	Room @ slot	Preferred	Note
T1	Auditorium @ 09:00	09:00	✓ in preference
T2	Canyon @ 09:00	10:30	soft miss
T3	Mesa @ 09:00	10:30	soft miss
T4	Canyon @ 09:00	13:00	soft miss
T5	Auditorium @ 09:00	13:00	soft miss

T6	Mesa @ 10:30	10:30	✓ in preference

- Hard = 5: room: {T1↔T5}@Auditorium, {T2↔T4}@Canyon → 2; speaker: {T1↔T3} Ada @09:00 → 1; audience: {T1↔T2}, {T3↔T5} @09:00 → 2. Conflicted = {T1,T2,T3,T4,T5} (T6 clean).
- Soft = 5: preferred misses T2,T3,T4,T5 → 4; track spread — AI uses {Auditorium, Mesa} → 2−1 = 1; Systems uses {Canyon} → 0; Practice → 0. Total 5.

3. Min-Conflicts

[Explain conflicted-talk selection, lexicographic candidate scoring, treatment of the current value, seeded tie breaking, budget, stopping, and metrics.] The agent must recompute the conflicted talk section every time that it makes a change, so that it has an up to date list. Candidates are scored based on hard violations first with soft penalties being the tiebreaker. This is done by storing those two values as a tuple and then comparing them. Every candidate is constructed by calling move on the current schedule with the possible change, so that it can get scored. The talk's current placement is omitted from the options so that the agent will change the schedule with every step when a valid step exists. In case of a tie a seeded random choice is picked so that we can remain deterministic while not getting stuck in cycles of always choosing the first found. I did not get around to implementing this in my code. The budget bounds the agent's moves and halts immediately when reached. Otherwise the agent stops as soon as a schedule with 0 hard conflicts is found and it returns that schedule. The metrics for min-conflicts include the number of edits it makes to the schedule, the number of evaluator calls, the runtime, and the length of the trace. Again because of the seeded random the same scenario and seed will always produce an identical schedule and trace.

4. LLM Tool Policy

Exact model tag: [Confirm `Gemma-4-26B-A4B-it-oQ4e-mtp`]
Endpoint category: [Confirm course OpenAI-compatible service; do not record the API-key value]
Temperature: [ ]
Seed support and value: [ ]

Prompt and Tool Schema

[Give the exact system/user prompt or a complete reproducible appendix reference, the JSON schema, duplicate-key rejection, bounded history policy, zero-history behavior, and parsing rules.]

Validation and Failure Handling

[Explain malformed, invalid, repeated, and client-failure handling. State that calls are counted before attempts, only exception types are retained, ordinary failures consume budget and preserve state, and `BaseException` is not caught. Explain why model proposals do not prove feasibility.]

5. Controlled Experiment

Paired command (`--compare --live`): [ ]
Utah Tech Tailnet connection confirmed: [Yes]
Scenarios: [Confirm all four]
Seeds: [Confirm 0, 1, 2]
Budget: [Confirm 20 calls/moves per run]
Machine/runtime context: [ ]

Method	Runs	Feasible	Rate	Mean final hard	Successful runs	Mean soft among successes	Mea edi
Min-conflicts	12	[ ]	[ ]	[ ]	[ ]	[ ]	

LLM policy	12	[ ]	[ ]	[ ]	[ ]	[ ]	

Method-specific metric	Mean per applicable run	Total	Denominator
Min-conflicts evaluator calls	[ ]	[ ]	12 runs
LLM model calls	[ ]	[ ]	12 runs
LLM malformed requests	[ ]	[ ]	12 runs
LLM invalid requests	[ ]	[ ]	12 runs
LLM repeated requests	[ ]	[ ]	12 runs
LLM client failures	[ ]	[ ]	All 12 runs, including failure runs

Do not discard request or client failures, and retain affected runs in every all-run denominator. Soft penalty is conditional on success; state the successful-run denominator rather than treating failures as zero or missing without explanation. Include the runner's explicit pair identifiers.

6. Representative Traces

Min-Conflicts Trace

[Scenario, seed, exact start, selected talks/moves, hard/soft values, stop.]

LLM Trace

[Same scenario/start/20-call-or-move budget if practical; retain

malformed, invalid, repeated, and client-failure outcomes.]

7. Analysis

Guarantees and Search Behavior

[Compare what each method guarantees and does not guarantee.]

State Versus Path and Invariants

[Explain final-state solutions, edit paths, structural validity, and feasibility.]

Prompt Sensitivity and Reproducibility

[Discuss prompt/model/seed effects and observed variance.]

Resource Fairness

[Discuss shared starts, the 20-call/move budget, move/evaluator access, and unequal evaluator/model costs.]

Constraint Verification

[Explain why tool validation and deterministic evaluation remain necessary.]

PEAS Alignment and External Validity

[Explain whether the experiment measures the performance criteria you defined, whether the supplied sensors and actuators are sufficient for the benchmark, and how your benchmark-versus-reality limitation

constrains the conclusions.]

8. Testing and Reproducibility

Test command/result: [ ]

[Describe hard-count, soft-penalty, invalid move, min-conflicts, duplicate-key, zero-history, client-failure, and scripted LLM tests. Explain why scripted tests are not empirical LLM evidence.]

9. AI-Assistance Disclosure

Tools: [opencode Big Pickle]
Material effect: [Cleaned up my evaluator code and helped me find what I was missing in the functions. Walked me through the differences between the random agent that I based my minconflict agents off of. Other debugging. Helped me make sure I hit all the points in my report]
Verification: [I ran tests for all the code it wrote, and wrote in my own words the answers to the report questions.]

Submission Checklist

- ☐ One accessible PDF named `lastname-firstname-conference-repair.pdf` with selectable text and semantic headings/tables.
- ☐ All four scenarios and seeds 0, 1, 2 appear in paired results.
- ☐ Exact model configuration, prompts, failures, and denominators are present.
- ☐ Both representative traces are included.
- ☐ PEAS, environment classifications, interaction cycle, and implementation mapping are complete and consistent with the technical work.
- ☐ Private GitHub URL, submitted commit, and `fractal13` read

access are confirmed.
- ☐ Tests pass and no credentials, tokens, authenticated URLs, or secrets are present in the PDF or repository.
- ☐ AI-assistance disclosure is complete.